Regresión logística y k vecinos más cercanos (kNN)

Sesión 8 · Módulo 3: Modelos de clasificación

Sigmoide y log-loss · OvR / OvO / softmax · kNN y la maldición de la dimensionalidad · Métricas de clasificación

Datos simulados · scikit-learn

Plan de la sesión (≈50 min)

Qué veremos

- **A.** Regresión logística: de una recta a una probabilidad, con sigmoide y pérdida logarítmica (log-loss)
- **B.** Estrategias multiclase: uno contra el resto (OvR), uno contra uno (OvO) y softmax
- **C.** kNN: métricas de distancia, elección de k y la maldición de la dimensionalidad
- **D.** Métricas de clasificación: exactitud, precisión, sensibilidad (recall) y F1

Dos modelos muy distintos entre sí (uno "aprende una fórmula", el otro "memoriza y compara") para la misma tarea: predecir una **categoría**, no un número.

De dónde venimos: cerramos regresión, abrimos clasificación

Módulo 2 (sesiones 4–7): predecir un número

Sesión	Modelo
4	Lineal y polinomial
5	Ridge, Lasso, Elastic Net
6	SVR
7	Árbol de decisión

Módulo 3 (hoy): predecir una categoría

La variable objetivo y ya no es un número continuo (precio, consumo) sino una **clase** (spam/no-spam, especie, diagnóstico). Cambian el **modelo** y las **métricas**; se mantiene todo lo demás: partición de datos, `Pipeline`, validación cruzada.

Hoy: regresión logística y kNN. Las siguientes sesiones del módulo: Naive Bayes, árboles, SVM y redes neuronales para clasificación.

A1 · ¿Por qué no usar la regresión lineal para clasificar?

La tentación

Si $y \in \{0, 1\}$, ¿por qué no ajustar una recta $\hat{y} = w^\top x + b$ y redondear?

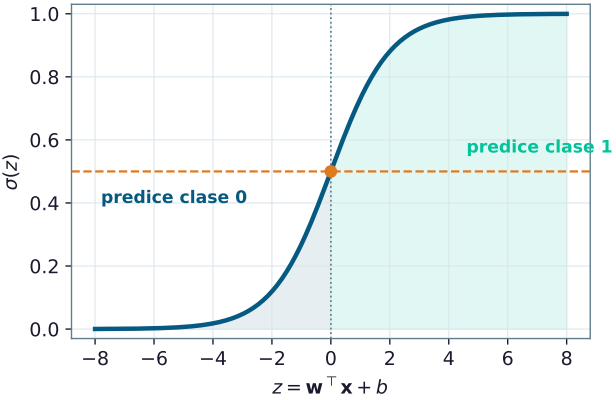
Los problemas

- La recta predice valores **fuera de** $[0, 1]$: $\hat{y} = 1.4$ o $\hat{y} = -0.3$ no son probabilidades.
- Un solo dato lejano **mueve la recta** y, con ella, el umbral de decisión.
- El error cuadrático **no castiga** igual todos los errores de clasificación.

La idea de hoy

Necesitamos una función que tome cualquier valor real y lo **"aplaste"** al intervalo $[0, 1]$, para interpretarlo como una probabilidad.

A2 · La función sigmoide



Definición

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z puede ser cualquier número real; $\sigma(z)$ siempre cae en $(0, 1)$.

Lectura

- $z = 0 \Rightarrow \sigma(z) = 0.5$: el punto de corte.
- $z \to +\infty \Rightarrow \sigma(z) \to 1$;
  $z \to -\infty \Rightarrow \sigma(z) \to 0$.

A3 · El modelo: regresión lineal + sigmoide

Regresión logística

$$z = w^\top x + b \qquad p(y{=}1 \mid x) = \sigma(z)$$

Es la **misma combinación lineal** de las sesiones 4–5, pasada por la sigmoide para convertirla en probabilidad.

Regla de decisión

$$\hat{y} = 1 \iff p(y{=}1 \mid x) \geq 0.5 \iff z \geq 0$$

Como $z = w^\top x + b$ es **lineal**, la frontera entre las dos clases es una **recta** (o un hiperplano, con más variables).

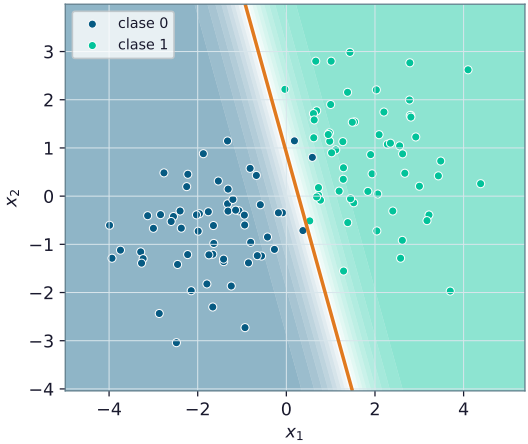
Para no confundir

"Logística" no ajusta una recta cuya **salida** sea y (como en la 4–5); ajusta la misma recta de siempre, pero su **salida** es z, no y:

$$\underbrace{z}_{\text{salida}} = w^\top \underbrace{x}_{\text{entrada}} + b$$

y la sigmoide $p = \sigma(z) = \dfrac{1}{1+e^{-z}}$ traduce esa z a probabilidad.

A4 · La frontera de decisión, en datos 2D



Lectura

- El color indica $p(y{=}1 \mid x)$: azul oscuro $\rightarrow$ cerca de 0, verde $\rightarrow$ cerca de 1.
- La línea naranja es la **frontera** donde $p = 0.5$: una recta.
- Exactitud del modelo en estos datos: **98.3 %**.

Con variables polinomiales (Sesión 4) la frontera puede curvarse, igual que con regresión polinomial.

A5 · ¿Por qué no entrenar minimizando el MSE?

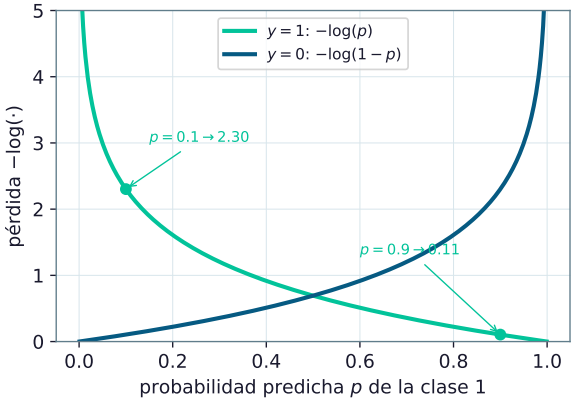
El problema

Si se usa $\frac{1}{n}\sum(\sigma(z_i) - y_i)^2$ como función de costo, **no es convexa** respecto a w, b: tiene mínimos locales donde el descenso de gradiente puede atorarse.

La solución

Se usa otra función de costo, pensada para probabilidades: la **pérdida logarítmica** (log-loss). Es **convexa**: un solo mínimo, el descenso de gradiente siempre lo encuentra.

A6 · La pérdida logarítmica (log-loss)



Por ejemplo

$$L(y,p) = -\big[y \log p + (1-y)\log(1-p)\big]$$

Si $y = 1$: $L = -\log p$ (castiga predecir p bajo).
Si $y = 0$: $L = -\log(1-p)$.

Lectura numérica ($y = 1$)

$p = 0.9 \Rightarrow L = 0.11$ (acertar con seguridad casi no cuesta). $p = 0.1 \Rightarrow L = 2.30$ (equivocarse con seguridad **cuesta mucho**).

A7 · Función de costo y entrenamiento

Costo total

$$J(w, b) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \sigma(w^\top x_i + b))$$

Convexa en (w, b): el descenso de gradiente (por lotes, estocástico o por minilotes, Sesión 4) converge al mínimo global.

En la práctica

scikit-learn no usa descenso de gradiente simple por defecto: usa optimizadores más rápidos (`lbfgs`, etc.). La idea –minimizar una función convexa– es la misma.

A8 · En scikit-learn: `LogisticRegression`

```python
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

modelo = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=1.0, max_iter=1000))
modelo.fit(Xtr, ytr)

modelo.predict(Xte)              # etiquetas 0/1
modelo.predict_proba(Xte)[:, 1] # probabilidades
```

Puntos clave

- `C` es el **inverso** de la regularización (como $1/\alpha$ en Ridge): **más** `C` = menos regularización = más complejo.
- Igual que SVR, conviene **escalar** las variables.
- `predict_proba` da la probabilidad, no solo la etiqueta.

A9 · Cómo interpretar los coeficientes

Es lineal en el *log-odds*

Invirtiendo la sigmoide: $\log \frac{p}{1-p} = w^\top x + b$. Cada w_j es el cambio en el **log-odds** por unidad de x_j.

La razón de momios: ¿de dónde sale e^{w_j}?

Como odds $= e^{\text{log-odds}} = e^{w^\top x + b}$, si $x_j \to x_j + 1$ (las demás fijas):

$$\text{odds}^{\text{nuevo}} = e^{w^\top x + b + w_j} = e^{w_j} \cdot \text{odds}^{\text{viejo}}$$

Por eso e^{w_j} **multiplica** los momios por cada unidad de x_j:

- $w_j > 0$ ($e^{w_j} > 1$): aumenta $P(\text{clase } 1)$.
- $w_j < 0$ ($e^{w_j} < 1$): la reduce.
- $w_j = 0$ ($e^{w_j} = 1$): sin efecto.

Ejemplo: filtro de spam

Variable	w_j	e^{w_j}
contiene “gratis”	1.8	6.05
# de enlaces	0.3	1.35
longitud del texto	-0.02	0.98

Leer la tabla: la palabra “gratis” **multiplica por 6** los momios de ser spam; cada enlace extra los multiplica por 1.35; cada carácter extra de longitud los multiplica por 0.98 (los reduce apenas).

Cuidado

Para comparar la **magnitud** de los coeficientes entre variables hay que **escalarlas** primero (igual que en Ridge/Lasso): si no, una variable en pesos y otra en años no son comparables.

A10 · Regularización en la regresión logística

El mismo costo de la Sesión 5, con log-loss

$$J(w,b) = \underbrace{\frac{1}{n}\sum_i L(y_i, p_i)}_{\text{log-loss}} + \frac{1}{C}\underbrace{\|w\|}_{\text{penalización}}$$

`C` es el **inverso** de α: `C` **grande** = poca penalización (más complejo); `C` **pequeño** = mucha penalización (más simple).

Tres sabores, igual que Ridge/Lasso/EN

`penalty`	**Efecto en los coeficientes**
`l2` (default)	los **encoge** hacia 0, sin eliminarlos
`l1`	pone algunos **exactamente en 0**: selección automática
`elasticnet`	combina l1 y l2 (`l1_ratio`)

```python
from sklearn.linear_model import (
    LogisticRegression)

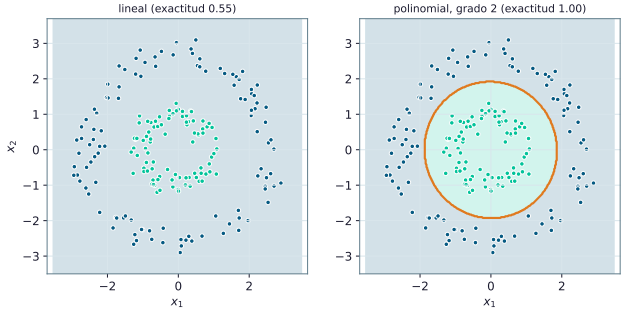
# L2, por defecto
m_l2 = LogisticRegression(
    penalty="l2", C=1.0)

# L1: requiere otro solver
m_l1 = LogisticRegression(
    penalty="l1", C=1.0,
    solver="liblinear")

# Elastic Net
m_en = LogisticRegression(
    penalty="elasticnet", C=1.0,
    l1_ratio=0.5, solver="saga")
```

`C` se elige igual que α en Ridge/Lasso: con `GridSearchCV` y validación cruzada. Con muchas variables (texto, miles de palabras) `l1` es especialmente útil: simplifica el modelo y lo hace más interpretable.

A11 · Regresión logística polinomial: fronteras curvas



Misma idea que en regresión (Sesión 4)

$z = w^\top x + b$ es lineal, así que la frontera ($p = 0.5$) **siempre** es una recta en las variables originales. Agregar variables polinomiales (x_1^2, x_2^2, $x_1 x_2$,...) antes de la sigmoide hace la frontera **curva** en el espacio original –aunque sigue siendo lineal en el espacio ampliado.

Lectura

Clases en círculos concéntricos: lineal apenas separa (exactitud **0.55**, casi al azar); con grado 2 separa **perfectamente** (**1.00**), porque $x_1^2 + x_2^2$ es justo lo que distingue un círculo del otro.

En código: `make_pipeline(PolynomialFeatures(degree=2), StandardScaler(), LogisticRegression(C=...))`.

Igual que en regresión: más grado = más flexible pero más riesgo de sobreajuste; por eso casi siempre va con **regularización** (diapositiva anterior) y con el grado elegido por validación cruzada.

Pausa para pensar (actividad 1, 3 min)

Trabaja de forma individual

1. Un modelo da $p = 0.97$ para un correo. ¿Qué etiqueta predice? Si en realidad **sí** era spam, ¿la pérdida es alta o baja? ¿Y si **no** lo era?
2. ¿Por qué la regresión logística no se entrena minimizando el error cuadrático medio (MSE)?
3. Si subes `C` en `LogisticRegression`, ¿el modelo se vuelve más simple o más complejo?

Respuestas: (1) predice spam (clase 1); si era spam, pérdida baja ($-\log 0.97 \approx 0.03$); si no lo era, pérdida muy alta ($-\log 0.03 \approx 3.51$). (2) El MSE con la sigmoide no es convexo: el descenso de gradiente puede quedar atrapado en un mínimo local; el log-loss sí es convexo. (3) Más complejo: `C` alto = poca regularización, igual que α bajo en Ridge/Lasso.

B1 · Del problema binario a varias clases

El problema

La sigmoide da **una** probabilidad para **dos** clases. ¿Qué hacemos si y toma $K > 2$ valores? (Por ejemplo: correo $\rightarrow$ spam / promocional / social / personal.)

Dos familias de estrategias

- **Descomponer** el problema en varios binarios: OvR, OvO.
- **Generalizar** directamente la sigmoide a K clases: softmax.

B2 · Uno contra el resto (One-vs-Rest, OvR)

Idea

Entrenar **un clasificador binario por clase**: "¿es de esta clase o de cualquier otra?". Con K clases, K modelos. Predicción: la clase cuyo clasificador dé **mayor** probabilidad.

Ejemplo numérico: un correo nuevo

Modelo	Distingue	$P(\text{clase})$
1	spam vs. {resto}	0.72
2	promocional vs. {resto}	0.18
3	social vs. {resto}	0.35
4	personal vs. {resto}	0.09

Cada modelo decide **por su cuenta**, así que las 4 probabilidades **no suman 1**. Gana la clase con mayor P: **spam** (0.72).

Costo

K modelos, cada uno entrenado con **todos** los datos. Escala bien cuando K es grande.

B3 · Uno contra uno (One-vs-One, OvO)

Idea

Entrenar **un clasificador binario por cada par** de clases. Con K clases, $\binom{K}{2} = \frac{K(K-1)}{2}$ modelos. Predicción: **votación** –cada modelo "vota" por una de sus dos clases– y gana la más votada.

Mismo correo: 6 modelos votan

Modelo (par)	Voto
spam–promo	spam
spam–social	spam
spam–personal	spam
promo–social	social
promo–personal	personal
social–personal	social

Conteo: spam $= 3$, social $= 2$, personal $= 1$, promo $= 0$. Gana **spam** (más votos).

Costo

Crece **cuadráticamente** con K (con 50 clases: 1225 modelos), pero cada modelo entrena con **menos** datos –solo 2 clases–, útil con clasificadores que no escalan bien en n (como SVM con kernel).

B4 · Softmax: la generalización directa

Un puntaje lineal por clase

Para cada clase k: $z_k = w_k^\top x + b_k$. La probabilidad de la clase k:

$$p(y{=}k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Las K probabilidades son positivas y **suman 1**.

Con $K = 2$, es la sigmoide

$\dfrac{e^{z_1}}{e^{z_0} + e^{z_1}} = \sigma(z_1 - z_0)$: softmax con dos clases **es** la regresión logística binaria.

Pérdida: log-loss multiclase (entropía cruzada), $-\log\big(p(y{=}\text{clase verdadera})\big)$: la misma idea de la diapositiva A6, con K clases.

B5 · Un ejemplo numérico: softmax con 4 clases

Puntajes lineales $z_k = w_k^\top x + b_k$ para un correo

Clase	z_k
spam	2.0
promocional	0.5
social	-1.0
personal	1.2

Probabilidades $p_k = e^{z_k} / \sum_j e^{z_j}$

Clase	p_k
spam	**0.58**
promocional	0.13
social	0.03
personal	0.26

$$\sum_k e^{z_k} = e^{2.0} + e^{0.5} + e^{-1.0} + e^{1.2} = 12.73$$

Lectura

Las 4 probabilidades **suman 1**. El puntaje más alto ($z = 2.0$, spam) no se traduce de forma lineal: $58\,\%$ de probabilidad, no el "doble" del siguiente. Predicción: la clase con mayor p_k, aquí **spam**.

B6 · Comparación y en scikit-learn

Cuándo usar cada una

Estrategia	# modelos	Cuándo
OvR	K	K grande; cualquier clasificador binario
OvO	$\binom{K}{2}$	clasificadores costosos en n (SVM)
Softmax	1 (conjunto)	opción nativa y moderna por defecto

```python
from sklearn.linear_model import LogisticRegression
# softmax (multinomial) es el default moderno:
modelo = LogisticRegression(max_iter=1000)
```

```python
from sklearn.multiclass import (
    OneVsRestClassifier,
    OneVsOneClassifier)
from sklearn.svm import SVC

ovr = OneVsRestClassifier(
    LogisticRegression())
ovo = OneVsOneClassifier(SVC())
```

`LogisticRegression` y `KNeighborsClassifier` manejan varias clases **de forma nativa**; no hace falta envolverlas.

Pausa para pensar (actividad 2, 3 min)

Trabaja de forma individual

Clasificas correos en 4 categorías: spam, promocional, social y personal.

1. ¿Cuántos modelos entrena OvR? ¿Y OvO?
2. ¿Por qué OvO deja de ser práctico con 50 categorías?
3. Si $K = 2$, ¿qué relación hay entre softmax y la sigmoide?

Respuestas: (1) OvR: 4 modelos. OvO: $\binom{4}{2} = 6$ modelos. (2) Con 50 clases, OvO necesitaría $\binom{50}{2} = 1225$ modelos: crece cuadráticamente. (3) Softmax con 2 clases se reduce exactamente a la función sigmoide.

C1 · La idea: “dime con quién andas. . . ”

k vecinos más cercanos

Para clasificar un punto nuevo: buscar sus k vecinos **más cercanos** en el conjunto de entrenamiento y **votar**: la clase mayoritaria entre ellos.

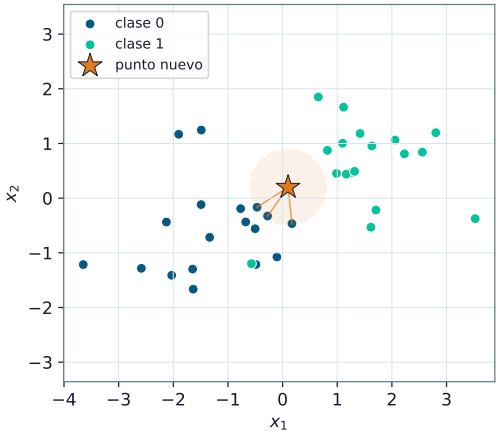
Un aprendiz “perezoso”

kNN **no ajusta parámetros**: en el “entrenamiento” solo guarda los datos. Todo el trabajo ocurre al **predecir**.

A diferencia de la regresión logística

No hay una fórmula (w, b) que aprender; no hay frontera lineal garantizada; la frontera se adapta a la forma local de los datos.

C2 · Un ejemplo con $k = 3$



Lectura

- La estrella es el punto nuevo a clasificar.
- El círculo punteado encierra a sus **3 vecinos más cercanos**.
- Los 3 son de **clase 0** $\Rightarrow$ predicción: **clase 0**.

Con empate en la votación (por ejemplo $k = 4$ y 2–2), scikit-learn desempata por la clase con los vecinos, en promedio, más cercanos.

Minkowski, la familia general

$$d(x, x') = \left(\sum_j |x_j - x'_j|^p \right)^{1/p}$$

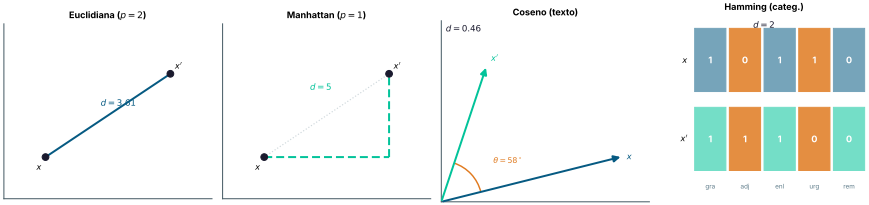
- $p = 2$: **Euclidiana**, "línea recta" (la más usada).
- $p = 1$: **Manhattan**, suma de diferencias absolutas ("cuadras de ciudad").

Otras distancias

Coseno: ángulo entre vectores, común en texto.
Hamming: cuenta diferencias, para variables categóricas.

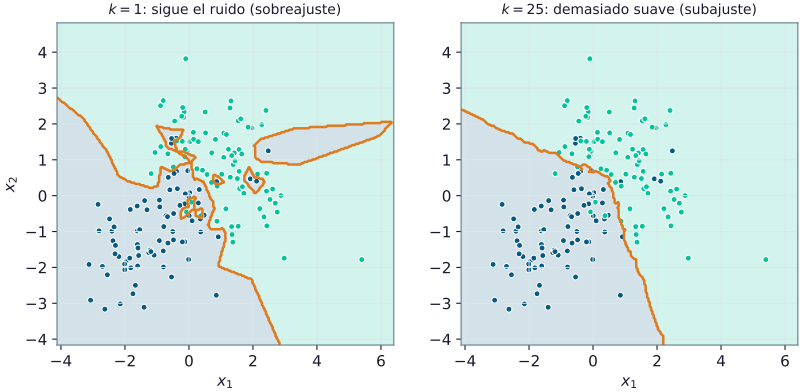
En scikit-learn: `metric="minkowski", p=2` por defecto (Euclidiana).

C3b · Las cuatro distancias, con un ejemplo



Mismo par de correos x, x' visto con cada métrica: Euclidiana y Manhattan sobre dos variables numéricas; Coseno sobre conteos de palabras; Hamming sobre atributos binarios.

C4 · Elegir k: sesgo y varianza, otra vez



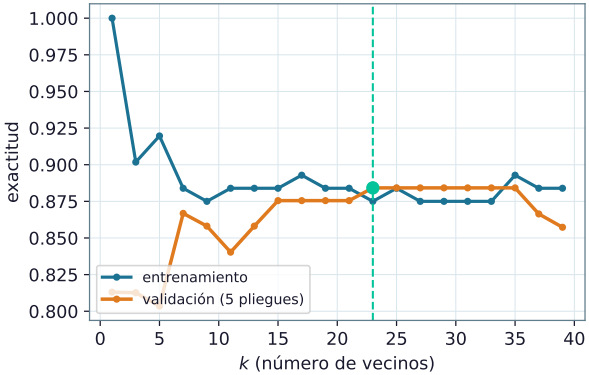
Lectura

- $k = 1$: la frontera sigue **cada punto**, incluido el ruido (varianza alta).
- $k = 25$: la frontera es casi una recta, **demasiado suave**

Al revés que `max_depth`

Aquí k **pequeño** es el modelo **complejo** y k **grande** es el **simple**: justo al revés que la profundidad de un árbol.

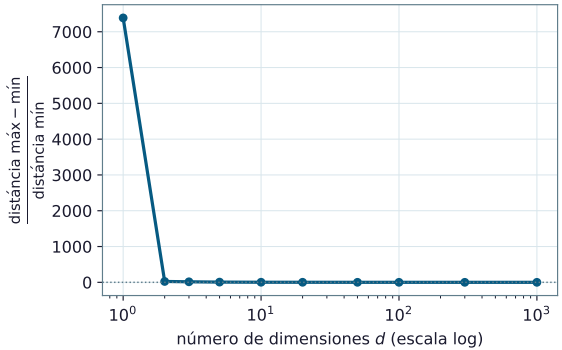
C5 · Elegir k con validación



Lectura

- $k = 1$: exactitud de entrenamiento **1.00** (memoriza) pero de validación solo **0.81**.
- El mejor equilibrio: $k = 23$, validación **0.88**.
- Con k muy grande, ambas curvas bajan: demasiada suavidad.

C6 · La maldición de la dimensionalidad



Qué mide la gráfica

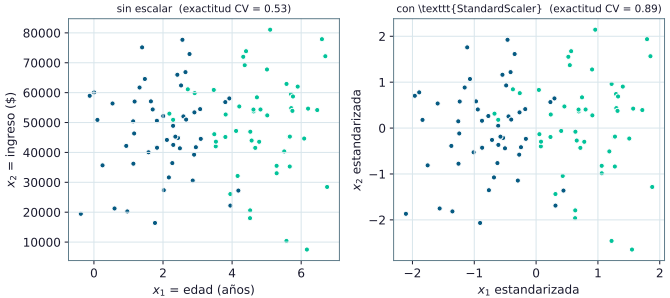
Para cada dimensión d, qué tan **más lejos** está el vecino más lejano que el más cercano, de un mismo punto de consulta.

Lectura

Con $d = 10$: el más lejano está $2.25\times$ más lejos que el más cercano (hay a quién distinguir). Con $d = 1000$: solo $0.13\times$ más lejos –**casi todos los puntos quedan a la misma distancia**, y "vecino más cercano" deja de significar algo.

Consecuencia práctica: con muchas variables, reducir la dimensión primero (selección de variables, Sesión 3, o PCA) antes de aplicar kNN.

C7 · Por qué escalar es obligatorio en kNN



Lectura

- Sin escalar, el **ingreso** (miles) domina la distancia; la **edad** casi no influye: exactitud **0.53**.
- Con `StandardScaler`, ambas variables pesan igual: exactitud **0.89**.

Más que en ningún otro modelo del curso

kNN mide **distancias**; una variable con escala mayor domina el cálculo aunque no sea más informativa. El escalado **siempre** va dentro del `Pipeline`.

C8 · En scikit-learn: KNeighborsClassifier

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

modelo = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(
        n_neighbors=23, metric="minkowski",
        p=2, weights="uniform"))
modelo.fit(Xtr, ytr)
modelo.predict(Xte)
```

Puntos clave

- `weights="distance"` pondera el voto por cercanía (más peso a los vecinos más próximos).
- El costo de **predecir** crece con n: compara contra todos los puntos guardados; bueno para datasets pequeños/medianos.

Pausa para pensar (actividad 3, 3 min)

Trabaja de forma individual

1. Con $k = 1$ obtienes exactitud de entrenamiento 1.00 y de validación 0.81. ¿Sesgo alto o varianza alta?
2. Tienes 500 variables y solo 200 datos. ¿Por qué kNN puede fallar? ¿Qué harías antes de aplicarlo?
3. Olvidaste escalar antes de usar kNN con variables en kilogramos y en pesos (MXN). ¿Qué pasa?

Respuestas: (1) varianza alta: memoriza el entrenamiento y se desinfla en validación. (2) maldición de la dimensionalidad: con tantas variables, todos los puntos quedan casi equidistantes; conviene reducir la dimensión (selección de variables o PCA) antes. (3) la variable en pesos (de mayor escala) domina la distancia; la de kilogramos casi no influye, igual que el ejemplo de edad e ingreso.

D1 · La matriz de confusión

Real \ Predicción	positivo	negativo
positivo	**VP** (TP)	**FN**
negativo	**FP**	**VN** (TN)

Vocabulario

VP: positivo, predicho positivo. FN: positivo, predicho negativo (*falsa alarma al revés*). FP: negativo, predicho positivo (*falsa alarma*). VN: negativo, predicho negativo.

Todas las métricas de hoy salen de aquí

Exactitud, precisión, sensibilidad y F1 son distintas formas de **resumir** estos cuatro números.

D2 · Exactitud (accuracy): cuándo engaña

Definición

$$\text{exactitud} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{VN} + \text{FP} + \text{FN}}$$

Proporción de predicciones correctas, positivas o negativas.

La paradoja de la exactitud

1000 correos, solo 50 son spam (5 %). Un modelo que **siempre** predice "no-spam" logra **95 % de exactitud**... sin detectar un solo spam.

Conclusión

Con clases **desbalanceadas** (común en spam, fraude, enfermedades raras) la exactitud sola no basta: hace falta mirar qué pasa con cada clase por separado.

D3 · Precisión y sensibilidad (recall)

Precisión

$$\text{precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

De lo que **predije** positivo, ¿cuánto acerté? Importa cuando un **falso positivo** es costoso.

Sensibilidad (recall)

$$\text{recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

De los positivos **reales**, ¿cuántos encontré? Importa cuando un **falso negativo** es costoso.

Hay un compromiso entre ambas

Filtro de spam: un falso positivo manda un correo importante a la basura $\Rightarrow$ se prioriza **precisión**. Diagnóstico médico: un falso negativo deja un caso sin detectar $\Rightarrow$ se prioriza **recall**.

D4 · F1: un solo número que combina ambas

Media armónica de precisión y recall

$$F_1 = 2 \cdot \frac{\text{precisión} \times \text{recall}}{\text{precisión} + \text{recall}}$$

Útil cuando se necesita **un solo** número para comparar modelos.

¿Por qué armónica y no promedio simple?

La armónica **castiga** que una de las dos sea baja. Precisión $= 1.0$ y recall $= 0.1$ dan promedio 0.55 pero $F_1 = 0.18$: un modelo así **no** es bueno, aunque el promedio simple lo disimule.

D5 · Un cálculo completo, paso a paso

100 correos de prueba (20 spam reales)

	pred. spam	pred. no-spam
spam real	VP $= 15$	FN $= 5$
no-spam real	FP $= 10$	VN $= 70$

Las cuatro métricas

$$\text{exactitud} = \frac{15+70}{100} = 0.85$$

$$\text{precisión} = \frac{15}{15+10} = 0.60$$

$$\text{recall} = \frac{15}{15+5} = 0.75$$

$$F_1 = 2 \cdot \frac{0.60 \times 0.75}{0.60+0.75} = 0.67$$

Buena exactitud (0.85) pero precisión y recall **moderados**: el filtro todavía deja pasar 5 spams y manda 10 correos buenos a revisión.

D6 · En scikit-learn: `classification_report`

```python
from sklearn.metrics import (
    confusion_matrix, classification_report)

ypred = modelo.predict(Xte)
print(confusion_matrix(yte, ypred))
print(classification_report(
    yte, ypred,
    target_names=["no-spam", "spam"]))
```

Puntos clave

- `classification_report` da precisión, recall y F_1 **por clase**, más promedios *macro* y *weighted*.
- Con clases desbalanceadas, revisar **siempre** la clase minoritaria, no solo el promedio.

Pausa para pensar (actividad 4, 3 min)

Trabaja de forma individual

1. Un filtro de spam tiene precisión 0.95 y recall 0.40. ¿Qué significa en la práctica? ¿Es buen filtro?
2. Un hospital usa un modelo para detectar una enfermedad grave. ¿Priorizarías precisión o recall? ¿Por qué?
3. Un modelo logra 98 % de exactitud en un problema con 1 % de casos positivos. ¿Es necesariamente bueno?

Respuestas: (1) cuando marca spam casi siempre acierta (pocas falsas alarmas) pero deja pasar 60 % del spam real (muchos falsos negativos): un filtro "conservador". (2) recall: no se puede permitir dejar pasar enfermos, aunque eso implique más falsas alarmas que revisar después. (3) no necesariamente: un modelo que siempre predice "negativo" ya tendría 99 % de exactitud sin detectar nada; hay que mirar precisión, recall y F_1.

Resumen de la sesión

Ideas clave

- **Regresión logística**: recta + sigmoide = probabilidad; se entrena minimizando el **log-loss** (convexo), no el MSE.
- **Multiclase**: OvR y OvO descomponen en binarios; **softmax** generaliza la sigmoide de forma directa.
- **kNN**: sin entrenamiento, vota entre los k vecinos más cercanos; k chico = varianza alta, k grande = sesgo alto.

Para llevarse

- kNN **necesita** variables escaladas y sufre la **maldición de la dimensionalidad**.
- **Exactitud** engaña con clases desbalanceadas; **precisión** y **recall** compiten entre sí; **F1** los resume en un número.
- Todo dentro de un `Pipeline`; validación cruzada para elegir C y k.

Siguiente sesión

Módulo 3: Naive Bayes y árboles de decisión para clasificación (sesión 9).

Actividad de laboratorio (individual)

Clasificación de correos spam con regresión logística y kNN

Cuaderno de Jupyter sobre un conjunto de correos etiquetados (spam / no-spam). Trabaja **de forma individual**.

1. Explora el conjunto de datos: ¿cuántos correos de cada clase? ¿está desbalanceado?
2. Divide en entrenamiento y prueba (estratificado); construye un `Pipeline` con `StandardScaler` + `LogisticRegression`, y ajusta `C` con `GridSearchCV`.
3. Construye otro `Pipeline` con `StandardScaler` + `KNeighborsClassifier`, y elige k con una curva de validación.
4. Compara ambos modelos con `classification_report` (exactitud, precisión, recall, F1) y su matriz de confusión.
5. ¿Cuál modelo comete más falsos positivos? ¿Cuál más falsos negativos?
6. Redacta una conclusión breve: ¿cuál recomendarías para un filtro de spam real, y por qué, en términos de precisión y recall?

Entregable: el cuaderno con la comparación y la conclusión justificada.



¿Preguntas?

¡Pasemos a la actividad de clasificación de correos spam!